Perfect approach. Let's build your mental model first before touching any code.

The Big Picture — What NoteMind Actually Is

Think of NoteMind like a **smart librarian**.

You give the librarian a bunch of textbooks (your PDFs). The librarian reads them, memorizes the important parts, and when you ask a question, they find the relevant pages and give you an answer.

That's exactly what your app does — just with code instead of a person.

The Two Halves of the App

```
NoteMind
 ├── Frontend  (what the user sees — React)
 └── Backend   (the brain — FastAPI + AI)
```

Frontend is like the counter at the library — where you walk in, drop off books, and ask questions.

Backend is the back room — where the actual reading, indexing, and answering happens.

They talk to each other over HTTP — the frontend sends requests, the backend sends back responses.

The Backend File System — The Story

```
backend/
 ├── app.py        ← The front door
 ├── ingest.py     ← The receiving desk
 ├── pdf_loader.py ← The reader
 ├── embeddings.py ← The memorizer
 ├── vector_store.py ← The filing cabinet
 ├── retrieval.py  ← The searcher
 ├── llm.py        ← The answerer
 └── data/
    └── uploads/   ← The storage room
```

Here's the story of what happens when a user uploads a PDF and asks a question:

Story Part 1 — Uploading a Document

```
User drops a PDF
   ↓
 app.py        ← receives the file from the browser
   ↓
 ingest.py     ← saves it to data/uploads/
   ↓
 pdf_loader.py  ← opens the PDF, reads all the text,
                cuts it into small chunks
   ↓
 embeddings.py  ← converts each chunk into numbers
                (called vectors — this is how AI
                understands text)
   ↓
 vector_store.py ← saves all those number-vectors
                into a FAISS index (a special
                searchable filing cabinet)
```

Story Part 2 — Asking a Question

```
User types a question
   ↓
 app.py        ← receives the question from the browser
   ↓
 embeddings.py  ← converts the question into numbers too
   ↓
 retrieval.py   ← searches the FAISS filing cabinet
                for chunks that are similar to
                the question
   ↓
 llm.py        ← takes those relevant chunks +
                the original question, sends them
```

```
                to an AI model, gets an answer back
  ↓
app.py        ← sends the answer back to the browser
```

The Frontend File System — The Story

```
frontend/src/
 ├── App.tsx          ← The map (decides which page to show)
 ├── pages/
 │   ├── Index.tsx      ← Landing page
 │   ├── Dashboard.tsx   ← Where you manage your documents
 │   ├── Chat.tsx       ← Where you ask questions
 │   ├── Flashcards.tsx  ← Study mode
 │   └── Compare.tsx     ← Compare two documents
 ├── components/
 │   ├── ChatWindow.tsx  ← The actual chat box
 │   ├── DocUploader.tsx ← The file upload box
 │   ├── AppSidebar.tsx  ← The left navigation bar
 │   └── ui/          ← Reusable buttons, cards etc
 └── lib/
     └── dummyData.ts    ← Fake data used while backend isn't ready
```

The frontend is like a building with multiple rooms. App.tsx is the hallway that decides which room to take you to based on the URL.

How Frontend and Backend Connect

```
Frontend (React)        Backend (FastAPI)
   │                    │
   │ POST /api/upload ──────→│ app.py receives file
   │                    │
   │ POST /api/query ──────→│ app.py receives question
   │                    │
   │ ←────── JSON response ──│ sends answer back
```

The frontend never touches the AI or files directly — it just sends HTTP requests to the backend and displays whatever comes back.

Current State of the Project

Part	Status
Frontend UI	✅ Built and working
Backend server	✅ Running on port 8000
File upload	✅ Working
PDF reading + chunking	✅ Working
Embeddings + FAISS	✅ Working
Query endpoint	☐ Returns dummy answer only
Real AI answers	☐ Not wired up yet

That's the full architecture. When you're ready, just say **"let's start file by file"** and we'll go through each one line by line starting with app.py — the front door of your backend.